DEPSTAR
Department of Computer Engineering
Charotar University of Science and Technology
Devang Patel Institute of Advance Technology and Research
Department of Computer Engineering

Practical 3: Sorting Algorithms: Comparison-Based, Counting-Based, and Digit-Based Strategies

➔ Practical Aim

To implement and compare multiple sorting algorithms and develop the ability to reason about which sorting strategy suits a given situation.

Problem 1

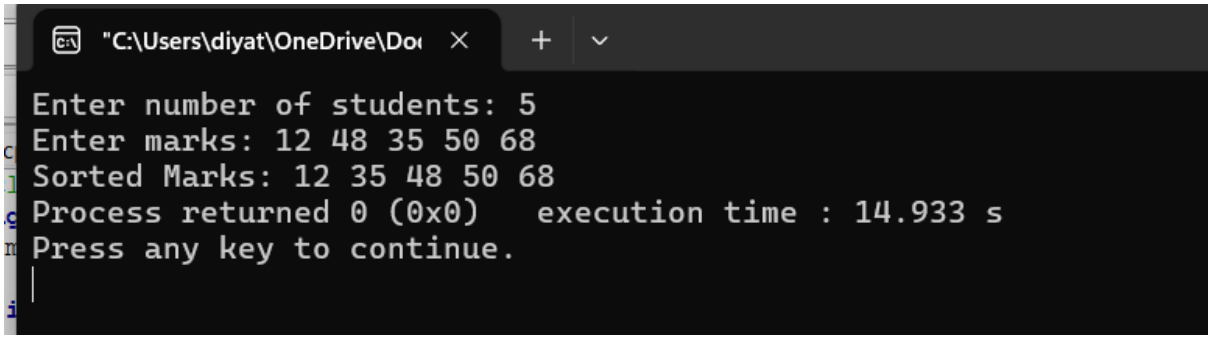
A teacher has a stack of student answer sheets with marks written on them and needs to arrange them in order before entering grades. She tries three different methods: in the first, she repeatedly compares adjacent sheets and swaps them if they are out of order; in the second, she finds the lowest-marked sheet each time and places it at the front; in the third, she picks each sheet one by one and inserts it into its correct position among the already-arranged sheets. Implement all three methods and for each one, print the sorted order of marks. Describe how each of the three methods works in your own words. If the sheets were already in order before the teacher started, which method would finish the fastest and why?

CODE(bubble sort)

```
#include <iostream>

using namespace std;

int main()

{

    int n;

    cout << "Enter number of students: ";

    cin >> n;

    int marks[n];
```

```
    cout << "Enter marks: ";
    for(int i=0;i<n;i++)
        cin>>marks[i];
    for(int i=0;i<n-1;i++)
    {
        for(int j=0;j<n-i-1;j++)
        {
            if(marks[j]>marks[j+1])
            {
                int temp=marks[j];
                marks[j]=marks[j+1];
                marks[j+1]=temp;
            }
        }
    }
    cout<<"Sorted Marks: ";
    for(int i=0;i<n;i++)
        cout<<marks[i]<<" ";
    return 0;
}
```

OUTPUT

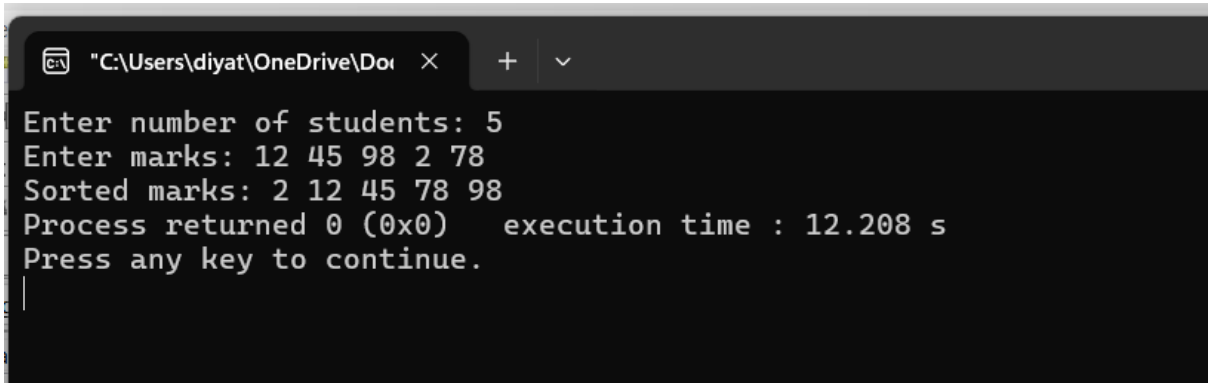


CODE(Selection)

```
#include <iostream>
using namespace std;
int main()
{
    int n;
    cout << "Enter number of students: ";
    cin >> n;
    int marks[100];
    cout << "Enter marks: ";
    for (int i = 0; i < n; i++)
    {
        cin >> marks[i];
    }
    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;
        for (int j = i + 1; j < n; j++)
        {
            if (marks[j] < marks[minIndex])
            {
                minIndex = j;
            }
        }
        int temp = marks[i];
        marks[i] = marks[minIndex];
        marks[minIndex] = temp;
```

```
    }
    cout << "Sorted marks: ";
    for (int i = 0; i < n; i++)
    {
        cout << marks[i] << " ";
    }
    return 0;
}
```

OUTPUT

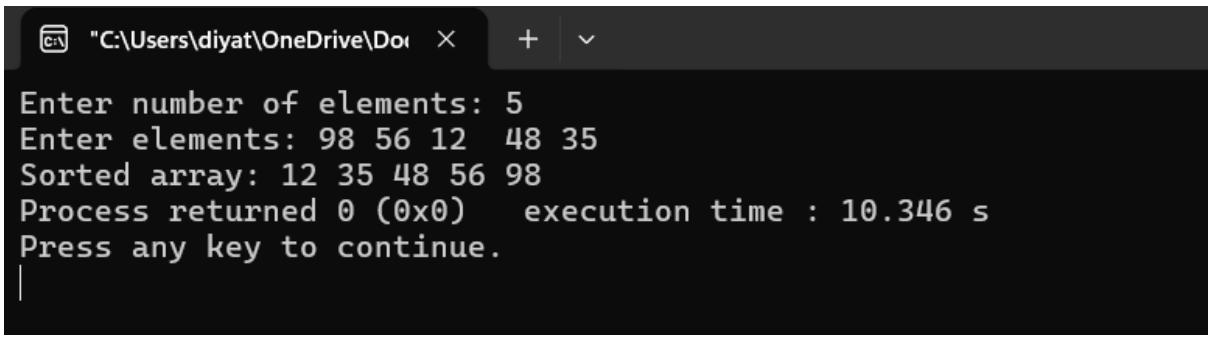


CODE(Insertion)

```
#include <iostream>
using namespace std;
int main()
{
    int n, a[100];
    cout << "Enter number of elements: ";
    cin >> n;
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
    {
```

```
        cin >> a[i];
    }
    for (int i = 1; i < n; i++)
    {
        int key = a[i];
        int j = i - 1;

        while (j >= 0 && a[j] > key)
        {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
    {
        cout << a[i] << " ";
    }
    return 0;
}
```

OUTPUT



The program displays the marks arranged in ascending order using Bubble Sort, Selection Sort, and Insertion Sort.

Conclusion:
Thus, Bubble Sort, Selection Sort, and Insertion Sort were successfully implemented to arrange the marks in ascending order. The working of all three sorting techniques was understood and compared.

Problem 2

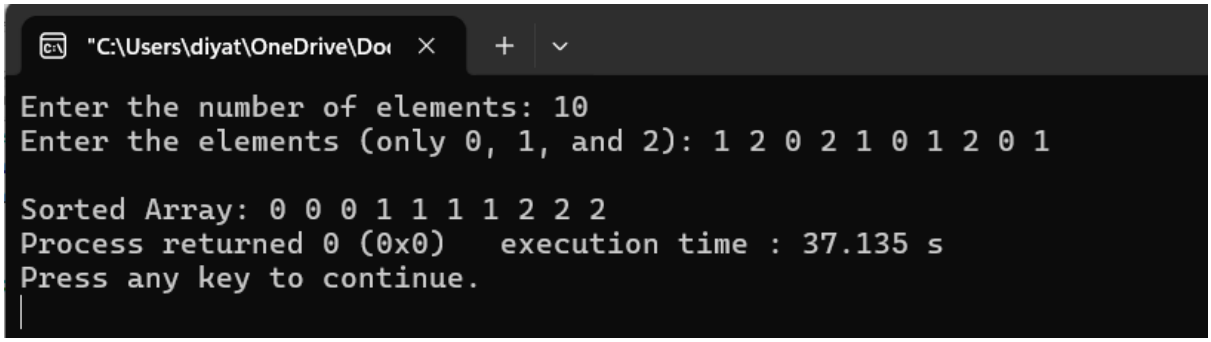
A paint shop has buckets labeled with one of three colour codes — 0, 1, or 2 — but they are stored in a random order. The shop owner wants all 0s together, then all 1s, then all 2s, without using any extra storage. Given the list of colour codes, rearrange them in place and print the result.Describe the approach you used. Does your solution make more than one pass through the list?Can you think of a way to do it in exactly one pass?

CODE

```
#include <iostream>

using namespace std;

int main()

{

   int n;

   cout << "Enter the number of elements: ";

   cin >> n;

   int a[n];

   cout << "Enter the elements (only 0, 1, and 2): ";

   for(int i = 0; i < n; i++)

   {

      cin >> a[i];

   }

   int low = 0;

   int mid = 0;
```

```
    int high = n - 1;
    while(mid <= high)
    {
        if(a[mid] == 0)
        {
            swap(a[low], a[mid]);
            low++;
            mid++;
        }
        else if(a[mid] == 1)
        {
            mid++;
        }
        else
        {
            swap(a[mid], a[high]);
            high--;
        }
    }
    cout << "\nSorted Array: ";
    for(int i = 0; i < n; i++)
    {
        cout << a[i] << " ";
    }
    return 0;
}
```

OUTPUT



The program rearranges the given colour codes so that all 0s come first, followed by all 1s and then all 2s, without using extra storage.

Conclusion:
Thus, the colour codes 0, 1, and 2 were successfully sorted in place. The approach demonstrates efficient rearrangement using constant extra space.

Key Questions / Analysis / Interpretation to be Evaluated During/After

Implementation

1. [Problem 1] One of the three methods can detect early that the list is already sorted and stop without doing any more work. Which method is it, how would you add that detection, and why can the other two not do the same?

Answer: Insertion Sort can detect that the list is already sorted because each element is compared with the previous element. If no shifting is required, it can finish early. Bubble Sort can also be modified with a swap flag to stop early if no swaps occur. Selection Sort normally cannot stop early because it still searches for the minimum element in the remaining part.

2. [Problem 2] If your solution makes two passes through the list, can you identify which decision in your logic forces the second pass? Is there a way to restructure that decision to eliminate it?

Answer: If the solution makes two passes, the second pass is usually required to place the elements into their final groups after counting the 0s, 1s, and 2s. This can be avoided by using the Dutch National Flag approach, which uses three pointers and sorts the array in exactly one pass.